Practical No. 1

Aim:

- Solving 2 water jug Problem: Write a program to solve the 2-water jug problem by manually providing moves from the user.
- Write a program to implement Missionaries and Cannibals problem. Constraints: three missionaries and three cannibals must cross a river using a boat which can carry at most two people, under the constraint that, for both banks, if there are missionaries present on the bank, they cannot be outnumbered by cannibals (if they were, the cannibals would eat the missionaries). The boat cannot cross the river by itself with no people on board.

Theory:

The Two water Jug Problem and the Missionaries and Cannibals Problem are examples of state space search problems in Artificial Intelligence. It demonstrates how complex situations can be solved using state space and constraints.

In the Water Jug Problem, the objective is to measure a specific quantity of water with two jugs of fixed capacity. Here the state tells the amount of water in the two jugs at an instance. First the initial state and goal state is decided and then the with the help of constraints and operations the problem is solved. The operations include (fill jug, empty jug, pour water from one to another). User is asked to give input until the goal is reached. When the user enters an input the program checks for the validity of input, and then updates the state accordingly, and checks whether the goal state is reached. This problem illustrates concepts like state representation, operators, and goal testing.

The Missionaries and Cannibals Problem is a river crossing problem. Here the state is represented by the no. of missionaries, the no. of cannibals, and the boat's position (right or left). The constraints are that the boat can carry up to two people and if missionaries are there, they should be more than or equal to cannibals. For the efficient and optimal path, we choose to apply Breadth First Search here using queue. Then in BFS the next states are checked that if they are valid or not with the help of the moves which we define. This gives the path of states. This problem highlights the importance of constraint satisfaction, state validation, and systematic search techniques such as BFS.

Together these problems help in understanding how Artificial Intelligence models real-world problems using states, transitions, and constraints to efficiently reach a desired goal.

Algorithm/Methodology:

<u>Two Water Jug Problem:</u>

1. Initialize two variables x and y for the amount of water in the jugs.
2. Take the goal state from the user i.e. the amount of water to be measured.
3. Repeat the following operations until the goal state is reached:
    - Fill jug x
    - Fill jug y
    - Empty jug x
    - Empty jug y
    - Pour water from x to y
    - Pour water from y to x

- Exit

4. After each operation print the value of x and y.
5. Display "Goal Reached" once the goal state is reached.

Missionaries and Cannibals:

1. Represent the state as tuple (M, C, B) and define the initial state and goal state.
2. Initialize the queue for BFS. Each queue element stores current state and the path taken to reach it.
3. Create visited set for already explored states.
4. While the queue is not empty:
   - Dequeue the first state and its corresponding path.
   - If the current state is goal state, return the path.
5. Define all the possible next states ensuring the constraint that the boat carries at most two people and is not empty.
6. For each generated state check the validity that the missionaries and cannibals are between 0 and 3 and on either bank if missionaries are present, they must be greater than or equal to cannibals.
7. If the next state is valid and not visited add it to the queue along with updated path.
8. If the queue is empty and the goal state is not reached return no solution.

Dataset Description – Two Water Jug Problem

Source:

The dataset is user-defined and dynamically generated during execution.

Attributes:

- Jug X capacity (cx): Maximum capacity of jug X (3 liters)
- Jug Y capacity (cy): Maximum capacity of jug Y (5 liters)
- x: Current amount of water in jug X
- y: Current amount of water in jug Y
- Goal: Desired amount of water to be measured

Each state is represented as a pair (x, y) indicating the current water levels in both jugs.

Size:

The dataset size is finite and small, limited by the capacities of the jugs.
For jugs of capacities 3 and 5, the total number of possible states is (3 + 1) × (5 + 1) = 24 states.

Dataset Description – Missionaries and Cannibals Problem

Source:

The dataset is synthetic and generated internally based on the classic Missionaries and Cannibals problem in Artificial Intelligence.

Attributes:

- M: Number of missionaries on the left bank
- C: Number of cannibals on the left bank
- B: Boat position (0 = left bank, 1 = right bank)

Each state is represented as a tuple (M, C, B).

Size:

The total possible states are limited by constraints:

- M = 0, 1, 2, 3

- $C = 0, 1, 2, 3$
- $B = 0, 1$

Maximum possible states = 4 × 4 × 2 = 32,
out of which only valid states (where missionaries are not outnumbered) are explored.

Implementation:

Two Water jug problem:

```
def two_water_jug(cx,cy,goal):
    # Initialize the amounts of water in jugs X and Y
    x, y = [0, 0]

    # Loop until either jug reaches the goal amount
    while (x != goal and y != goal):
        c = int(input("enter choice"))

        if c == 1:
            # Fill jug X to its full capacity
            x = 3
            print("Fill x")
        elif c == 2:
            # Fill jug Y to its full capacity
            y = 5
            print("Fill y")
        elif c == 3:
            # Empty jug X completely
            x = 0
            print("Empty x")
        elif c == 4:
            # Empty jug Y completely
            y = 0
            print("Empty y")
        elif c == 5:
            # Pour water from jug X to jug Y
            # Calculate available space in jug Y
            space = cy - y
            # Pour the minimum of water in X or space in Y
            pour = min(x, space)
            x = x - pour
            y = y + pour
            print("Pour x to y")
        elif c == 6:
            # Pour water from jug Y to jug X
            space = cx - x
            pour = min(y, space)
            x = x + pour
            y = y - pour
            print("Pour y to x")
```

```
        elif c == 7:
            # Exit the program
            print("Exit")
            break
        else:
            # Handle invalid input
            print("Invalid choice")

        # Display current state of the jugs after every operation
        print(x, y)

    # Goal state reached
    print("goal reached")

# Take goal input from user
print("Enter goal")
g = int(input())
two_water_jug(3, 5, g)
```

Missionaries and Cannibals problem :

```
# BFS function
def bfs():
    start = (3, 3, 0)   # Initial state: 3 missionaries, 3 cannibals, boat on left bank (0)
    goal = (0, 0, 1)   # Goal state: all missionaries and cannibals on right bank, boat on
right bank (1)

    # Queue stores tuples: (current_state, path_to_reach_state)
    queue = [(start, [start])]

    # Set to keep track of visited states to avoid cycles
    vis = set()

    while queue:
        # Remove first element from queue (FIFO for BFS)
        state, path = queue.pop(0)
        vis.add(state)

        # Check if current state is the goal
        if state == goal:
            return path

        # Generate all valid next states from current state
        for next_state in gen_nxtstate(state):
            # Add next state to queue if not visited already
            if next_state not in vis:
                queue.append((next_state, path + [next_state]))

    return None   # Return None if no solution exists
```

```python
# Function to generate all possible next states from a given state
def gen_nxtstate(state):
    nextstate = []
    mis, can, boat = state

    # Possible moves: (missionaries, cannibals) to move in boat
    move = [(1,0), (0,1), (1,1), (2,0), (0,2)]

    for m, c in move:
        # Skip invalid boat moves: cannot move 0 or more than 2 people
        if m + c == 0 or m + c > 2:
            continue

        # If boat is on left bank (0), move people to right
        if boat == 0:
            ns = (mis - m, can - c, 1)
        # If boat is on right bank (1), move people to left
        else:
            ns = (mis + m, can + c, 0)

        # Add state if it satisfies the constraints
        if valid(ns):
            nextstate.append(ns)

    return nextstate


# Function to check if a state is valid
def valid(state):
    mis, can, boat = state

    # Missionaries and cannibals must be within valid bounds
    if mis < 0 or mis > 3 or can < 0 or can > 3:
        return False

    # On left bank checking constraint for missionaries
    if mis > 0 and mis < can:
        return False

    # On right bank checking constraint for missionaries
    if 3 - mis > 0 and 3 - mis < 3 - can:
        return False

    return True
print(bfs())
```

Output / Result:

Water jug problem:

```
Enter goal
4
enter choice2
Fill y
0 5
enter choice6
Pour y to x
3 2
enter choice3
Empty x
0 2
enter choice6
Pour y to x
2 0
enter choice2
Fill y
2 5
enter choice6
Pour y to x
3 4
goal reached
```

Missionaries and cannibals problem:

```
[(3, 3, 0), (2, 2, 1), (3, 2, 0), (3, 0, 1), (3, 1, 0), (1, 1, 1), (2, 2, 0), (0, 2, 1), (0, 3, 0), (0, 1, 1), (1, 1, 0), (0, 0, 1)]
```

Result Analysis:

Two Water Jug Problem

The program takes the user's moves and updates the state of both jugs after each operation. The output shows the current amount of water in jug X and jug Y at every step. By performing operations like fill, empty, or pour, the program guides the user to reach the goal amount. Once the goal is achieved in either jug, the program prints "Goal Reached", indicating the successful completion of the task.

Missionaries and Cannibals Problem

The BFS algorithm generates a sequence of valid states from the initial state to the goal state. The output displays the path showing the number of missionaries, cannibals, and boat position at each step. It ensures that the constraints are never violated. The result provides an optimal sequence of moves to safely transfer all missionaries and cannibals to the other bank. This illustrates how BFS explores the state space efficiently while maintaining constraints.

Questions:

1.Why is BFS preferred over DFS here?

BFS is preferred because it guarantees finding the shortest sequence of moves while exploring all valid states level by level.

2.What happens if boat capacity is increased by 3?

The problem becomes easier, allowing more people to cross at once, reducing the number of moves needed.

3. Can the water jug problem be solved instead of manual input? How?

Yes, it can be solved automatically using BFS or DFS to explore all possible jug states and find a sequence of moves to reach the goal.

4. How do constraints prune invalid state?

Constraints discard states where missionaries are outnumbered or jug limits are exceeded, preventing unsafe or impossible moves.

Conclusion:

Both the Two Water Jug problem and the Missionaries and Cannibals problem demonstrate how Artificial Intelligence uses state space representation and constraints to solve complex problems systematically. The Water Jug problem illustrates interactive problem-solving using user-defined operations to reach a goal state, while the Missionaries and Cannibals problem shows how Breadth-First Search (BFS) explores all valid states to find an optimal solution without violating constraints. Together, these problems highlight the importance of state transitions, valid moves, and systematic search strategies in solving real-world problems efficiently and logically.